

Physics IDE — Technical Architecture, Deployment & Institutional Integration

Version: 1.0 · **Date:** March 2026

Document type: Technical Reference

Audience: Developers, System Administrators, ICT Staff, e-Learning Coordinators

Table of Contents

1. [Executive Summary](#)
 2. [System Architecture](#)
 3. [Technology Stack & Rationale](#)
 4. [Complete Source Map](#)
 5. [Simulation Execution Pipeline](#)
 6. [Build & Deployment](#)
 7. [Security Architecture](#)
 8. [Performance Profile](#)
 9. [Sustainability & Maintenance](#)
 10. [UWC iKamva Sakai Integration](#)
 11. [Data Privacy & POPIA](#)
 12. [Accessibility](#)
 13. [Recommended Roadmap](#)
 14. [Appendix A — Constants Reference](#)
 15. [Appendix B — vercel.json Template](#)
 16. [Appendix C — LTI 1.3 Registration Checklist](#)
-

1. Executive Summary

Physics IDE is a **zero-install, browser-based 3D physics simulation environment** built as a single-page React application. It requires no backend server, no database, and no user accounts. Students construct and run physically accurate 3D simulations using a visual block editor (Google Blockly) or a Python

code editor (Monaco), with results rendered live in a WebGL viewport powered by GlowScript/VPython 3.2.

Key technical facts:

- React 18 SPA, bundled by Create React App (Webpack 5)
- Build output: ~249 kB gzip (JS) + ~9 kB (CSS)
- Deployment: Vercel static hosting (zero server maintenance)
- No backend, no database, no user accounts
- All student data stored exclusively in browser `localStorage`
- Three CDN-loaded libraries: Blockly v11, Monaco 0.45, GlowScript 3.2
- Fully HTTPS; compatible with Sakai Web Content tool (iframe embedding)

2. System Architecture

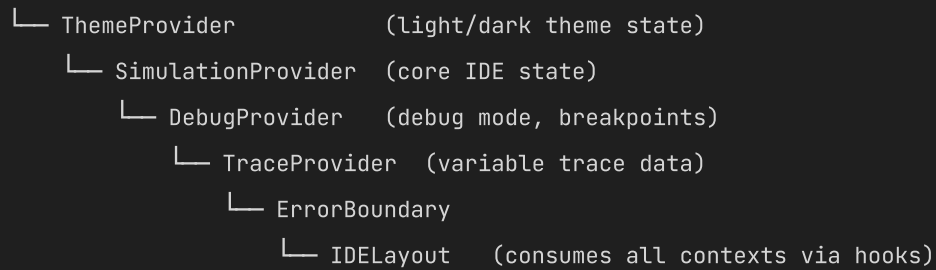
2.1 High-Level Overview

```
Browser
|
├─ React SPA (https://physics-ide.vercel.app)
|   ├── App.js           – Context provider tree
|   ├── IDELayout.js     – Render orchestrator
|   ├── hooks/           – All business logic
|   ├── components/      – Pure rendering
|   └─ utils/            – Stateless utilities
|
├─ GlowScript iframe     – Sandboxed VPython runtime
|   └─ https://www.glowscript.org (external, different origin)
|
└─ CDN resources (loaded at startup)
    ├── https://unpkg.com/blockly@11/...
    └─ https://unpkg.com/monaco-editor@0.45/...
```

2.2 Context + Hooks Architecture

Physics IDE uses a strict **provider / consumer separation**:

App.js



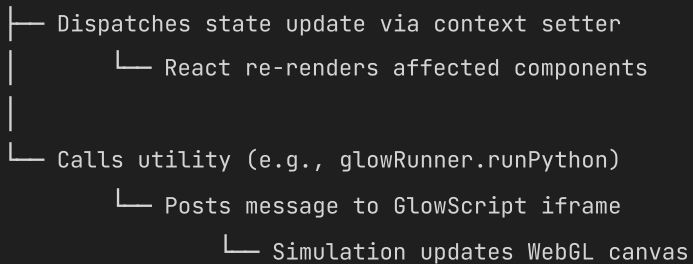
Rule: Components never call `useState` for shared state. All shared state lives in contexts. All actions live in hooks. Components receive only the data and callbacks they need via props.

2.3 Data Flow

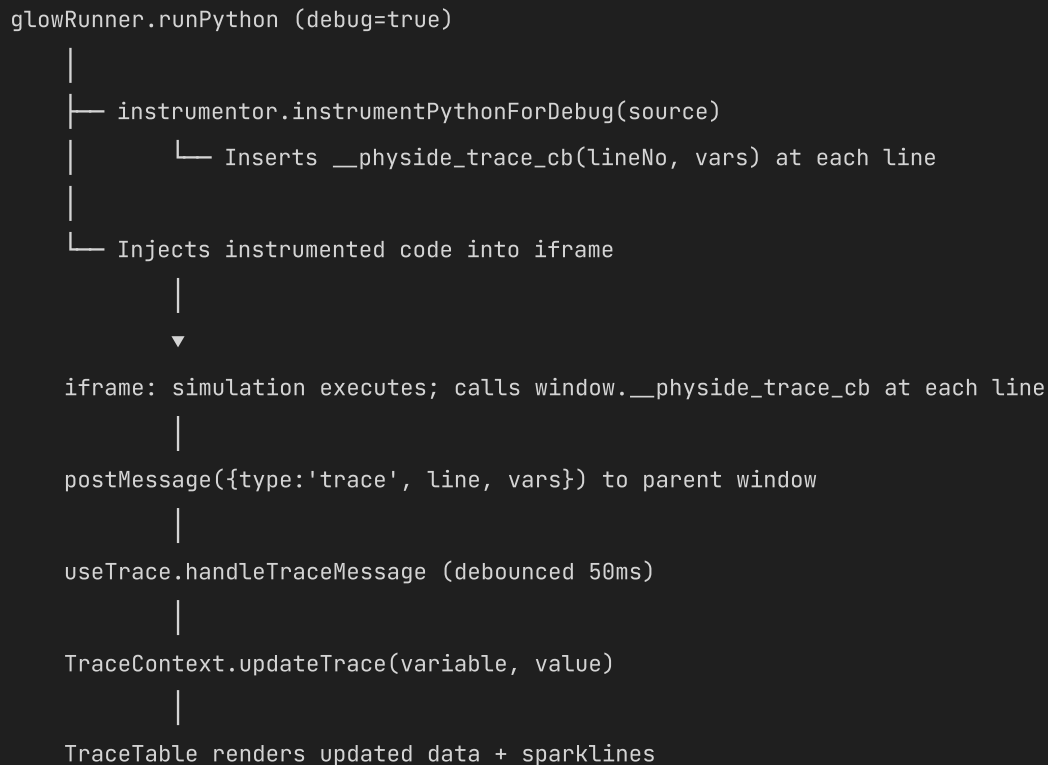
User action



Hook handler (e.g., `useSimulation.handleRun`)



2.4 Debug Mode Data Flow



2.5 Editor ↔ Simulation Bridge

The block editor and code editor share the same Python source string stored in `SimulationContext`. When in Blocks mode, the `BlocklyWorkspace` component calls `generatePythonFromWorkspace(workspace)` on every workspace change and syncs the result into the context. When in Code mode, the Monaco editor directly edits the same Python string.

3. Technology Stack & Rationale

3.1 Core Framework — React 18.3.1

Why React?

- Component-based architecture naturally maps to the IDE's panel structure.
- Context API + hooks provide clean state management without external dependencies.
- React 18 concurrent features (Suspense, `startTransition`) are available for future optimisation.
- Massive ecosystem; easy to find contributors.

Trade-offs:

- React's virtual DOM adds overhead not needed for purely static panels, but this is negligible at the IDE's scale.

3.2 Visual Block Editor — Google Blockly v11

Why Blockly?

- Industry standard for visual programming in education (used in Scratch, MIT App Inventor, code.org).
- Extensible: custom block types defined in JavaScript with JSON shape descriptors + code generator functions.
- Built-in undo/redo, copy/paste, block search, variable management, function definitions.
- Dark/light theming API available.
- v11 loaded from CDN (`unpkg.com/blockly@11`) rather than bundled — keeps React bundle lean.

Custom block system: All VPython-specific blocks are defined in `blocklyGenerator.js`. Each block definition includes:

1. A JSON shape descriptor (inputs, fields, colours, connections).
2. A Python generator function that produces VPython code from the block's field values and child blocks.

3.3 Code Editor — Monaco Editor 0.45.0

Why Monaco?

- Same editor engine as VS Code — students transitioning to professional tools find it familiar.
- Rich API for syntax highlighting, IntelliSense, keybindings.
- Loaded from CDN to avoid adding ~3 MB to the npm bundle.

3.4 Physics Simulation Runtime — GlowScript / VPython 3.2

Why GlowScript?

- VPython is the dominant language for introductory computational physics (used in Matter & Interactions, Physics for Scientists and Engineers).
- GlowScript is the browser-native runtime for VPython — no Python installation required.
- WebGL 3D rendering with built-in camera controls, lighting, material properties.
- The runtime runs in a **sandboxed cross-origin iframe** — the student's simulation code is isolated from the IDE's JavaScript context.

Version pinning: The `GlowScript 3.2 VPython` header is hard-coded into every generated program. If GlowScript releases a breaking change, the IDE continues to work against the pinned version served from `glowscript.org`.

3.5 PDF Generation — jsPDF 4.1.0 + html2canvas 1.4.1

- `html2canvas` captures the block canvas as a raster image.
- `jsPDF` assembles the PDF with the captured image and/or syntax-highlighted code text.
- Both are npm-bundled (not CDN) — included in the 249 kB bundle.

3.6 Build Tool — Create React App 5.0.1

CRA wraps Webpack 5 and provides:

- Zero-config build pipeline.
- Environment variable injection via `.env`.
- Code splitting and tree-shaking.
- ESLint integration.

CRA maintenance status: CRA is in maintenance mode (no new features). The recommended migration path is to **Vite**, which offers:

- ~10× faster cold starts.
- Native ESM dev server.
- Smaller configuration surface.

Migration is low-risk and low-effort (2–4 hours); recommended for the next major version.

4. Complete Source Map

src/

└─ App.js

Purpose: Provider wrapper only. Renders ThemeProvider > SimulationProvider >
DebugProvider > TraceProvider > ErrorBoundary > IDELayout.

LOC: ~30

└─ index.js

Purpose: React DOM root; attaches to <div id="root"> in index.html.

└─ styles.css

Purpose: All global styles (~800 LOC). VS Code-inspired dark/light theme.
CSS custom properties on :root used for theme switching.

└─ constants/index.js

Exports: STORAGE_KEY, THEME_STORAGE_KEY, DEFAULT_PYTHON_CODE,
ZOOM_MIN/MAX/DEFAULT (15/100/50), SPLIT_MIN/MAX/DEFAULT (15/85/50),
TRACE_HISTORY_SIZE (60), TRACE_DEBOUNCE_MS (50),
HIGHLIGHT_DURATION_MS (250), AUTOSAVE_INTERVAL_MS (2000),
GLOWSCRIPT_HOST_ID ("glowsript-host")

└─ contexts/

└─ ThemeContext.js

State: isDark (bool)

Effects: sets data-theme="dark"|"light" on <html>; persists to localStorage

Exports: ThemeProvider, useTheme()

└─ SimulationContext.js

State: mode ("blocks"|"code"), running, pythonCode, blockXml,
showStart, showHelp, viewportHidden, beginnerMode,
zoom, splitPos, workspaceRef, filename

Effects: auto-save every 2s; restore from localStorage on mount

Exports: SimulationProvider, useSimulationContext()

└─ DebugContext.js

State: debugMode (bool), breakpoints (Set<string>), breakpointsRef (ref),
executingBlockId (string|null)

Exports: DebugProvider, useDebugContext()

└─ TraceContext.js

State: traceData (Map<name, {values[], current, delta, min, max}>),
recording (bool), recordingRef (ref), recordBufferRef (ref)

Logic: updateTrace keeps rolling 60-point history per variable

Exports: TraceProvider, useTraceContext()

hooks/

useLocalStorage.js

API: [value, setValue] = useLocalStorage(key, initialValue)

Notes: JSON serialize/deserialize; handles SSR safely

useTrace.js

API: { startRecording, stopRecording, clearTrace }

Internal: registers window.__physide_trace_cb; adds postMessage listener;
routes trace data to TraceContext; debounces at 50ms

useDebug.js

API: { handleEnterDebug, handleExitDebug, handlePause, handleResume,
handleStep, handleToggleBreakpoint }

Notes: handleExitDebug internally calls useTraceContext to stop recording

useSimulation.js

API: { handleRun, handleStop, handleReset, handleModeChange,
handleZoomChange, handleWorkspaceReady, handleWorkspaceChange,
handleStartMenuSelect, handleImport, handleClearWorkspace,
handleToggleViewport, handleToggleBeginnerMode }

Deps: useSimulationContext, useDebugContext, glowRunner, dialogService

useExport.js

API: { handleExportPy, handleExportBlocks, handleExportBlocksPdf,
handleExportCodePdf, handleExportScreenshot, handleCopyCode }

Deps: useSimulationContext, exportUtils, pdfExport, dialogService

useSplitPane.js

API: { splitPos, handleSplitDrag }

Notes: clamps at SPLIT_MIN/MAX (15-85%); fires Blockly resize event

components/

layout/

IDELayout.js

Purpose: Calls all 6 hooks; conditionally renders StartMenu,
DebugMode, or the main IDE shell (Toolbar + editors + canvas)

common/

ErrorBoundary.js

Purpose: Catches render errors; shows fallback UI instead of white screen

Toolbar.js

Props: 28 callbacks + 8 display state props

Renders: navigation, simulation controls, workspace actions,
export dropdown, view toggles, zoom slider, theme toggle

└─ BlocklyWorkspace.js

Props: initialXml, onWorkspaceReady, onWorkspaceChange, isDark, beginnerMode

Internal: injects Blockly CDN; calls defineCustomBlocksAndGenerator;
manages workspace lifecycle; provides BlockSearch and ReadOnlyBlockly

└─ CodeEditor.js

Wraps Monaco Editor; listens to CDN load; syncs value ↔ context

└─ GlowCanvas.js

Renders a sandboxed <iframe> with id=GLOWSCRIPT_HOST_ID;
glowRunner targets this element

└─ DebugMode.js

Full-screen overlay: side-by-side ReadOnlyBlockly + code panel;
pause/resume/step/record controls; breakpoint line markers

└─ TraceTable.js

Reads TraceContext; renders table with sparklines using Canvas 2D API;
pin, search, CSV export, clear

└─ StartMenu.js

Template card grid; filter (blocks/code/all); calls onSelect with
{ mode, code|xml } payload

└─ ModeToggle.js

Toggle switch with confirmation dialog when switching away from
a non-empty workspace

└─ HelpPage.js

Full-screen overlay; searchable index (SEARCH_INDEX constant);
section anchors; all educational content inline

└─ Icons.js

Pure SVG icon components: PlayIcon, StopIcon, BugIcon, TableIcon,
DownloadIcon, etc. Accepts { size } prop.

└─ VariableDialog.js

Custom modal UI; receives { type, message, defaultValue } from
dialogService; resolves a Promise with the user's response

└─ utils/

- └─ storage.js
 - Exports: storageGet(key), storageSet(key, value), storageRemove(key)
 - Notes: all operations wrapped in try/catch (private browsing may throw)
- └─ blockTemplates.js
 - Exports: BLOCK_TEMPLATES (array of {id, title, mode, xml})
 - Notes: pre-built Blockly XML snippets for starter templates
- └─ precodedExamples.js
 - Exports: EXAMPLES (array of {id, title, subtitle, description, code})
 - Includes: Projectile Motion, Spring-Mass Oscillator, Electric Field
- └─ blockly/
 - └─ traceRegistry.js
 - Exports: traceRegistry (mutable array), clearTraceRegistry()
 - Notes: populated during generatePythonFromWorkspace; consumed by glowRunner
 - └─ blocklyGenerator.js
 - Exports: defineCustomBlocksAndGenerator(Blockly),
 - generatePythonFromWorkspace(workspace),
 - BLOCK_CATALOGUE (array of {type, label, category, keywords}),
 - customConstantsRegistry (mutable array)
 - LOC: ~1,600
 - Block categories: Values, Objects, Motion, State, Control, Advanced, 3D Math
 - └─ index.js – Barrel re-export
- └─ runner/
 - └─ instrumentor.js
 - Exports: instrumentPythonForDebug(source) → { source, entries }
 - Algorithm: line-by-line AST-free transformer; inserts
 - __physide_trace_cb(lineNo, {varName: varValue, ...}) before
 - each executable statement; extracts all assigned variable names
 - └─ glowRunner.js
 - Exports: runPython(code, traceRegistry, debugMode),
 - stopPython(), pausePython(), resumePython(),
 - stepPython(), setBreakpoints(breakpoints)
 - Mechanism: queries DOM for iframe#glowscrip-host;
 - builds GlowScript HTML payload; sets iframe.srcdoc
 - └─ index.js – Barrel re-export
- └─ export/
 - └─ syntaxHighlighter.js

```
| Exports: tokenizePython(source) → [{type, text}]
| Token types: keyword, builtin, string, comment, number, operator, default
|
|— pdfExport.js
| Exports: exportBlocksPdf(workspaceEl), exportCodePdf(pythonCode)
| Deps: jsPDF, html2canvas, syntaxHighlighter
|
|— exportUtils.js
| Exports: exportPython(code, filename), exportBlocks(xml, filename)
| Mechanism: Blob URL + <a download> click trigger
|
|— dialogService.js
| Exports: registerDialogService(impl), prompt(msg, def),
|           alert(msg), confirm(msg), promptFileName(def)
| Pattern: service locator – VariableDialog registers itself on mount;
|           hooks call the registered service to show modals
|
|— index.js – Barrel re-export
```

5. Simulation Execution Pipeline

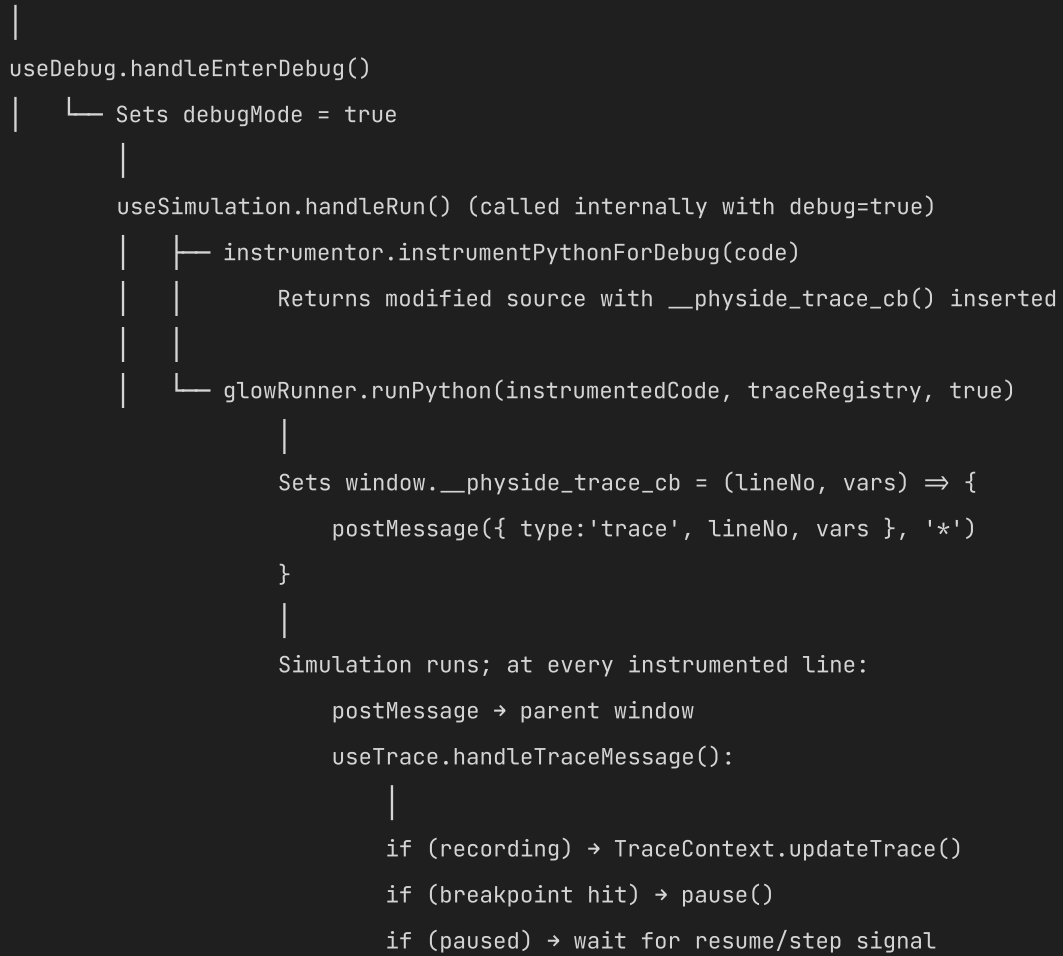
5.1 Normal Run

User clicks Run

```
|
| useSimulation.handleRun()
|   |─ Reads pythonCode from SimulationContext
|   |─ Sets running = true
|   └─ Calls glowRunner.runPython(code, traceRegistry, false)
|
| glowRunner builds HTML payload:
| <html>
|   <head>
|     <script src="glowscript.org/package/gspy.js"></script>
|   </head>
|   <body>
|     <div id="glowscript">
|       <script type="text/javascript">+GS 3.2 VPython
|         {user code here}
|       </script>
|     </div>
|   </body>
| </html>
|
| Sets iframe.srcdoc = payload
|
| GlowScript runtime:
|   - Parses VPython
|   - Converts to JavaScript (GlowScript's transpiler)
|   - Executes in iframe context
|   - Renders WebGL to <canvas> in iframe
```

5.2 Debug Run

User clicks Debug



5.3 Code Generation (Blocks → Python)

Any block change in Blockly workspace

```
|
| BlocklyWorkspace.onChange()
|   └─ generatePythonFromWorkspace(workspace)
|       |
|       | clearTraceRegistry()
|       |
|       | Blockly.Python.workspaceToCode(workspace)
|       |   For each connected block (depth-first):
|       |   Calls gen[block.type](block)
|       |   Returns Python string fragment
|       |   Builds traceRegistry entries for traced variables
|       |
|       | Returns complete Python string
|
| onWorkspaceChange(xml, pythonCode)
|
| SimulationContext.setPythonCode(pythonCode)
|
| SimulationContext.setBlockXml(xml)
```

6. Build & Deployment

6.1 Prerequisites

REQUIREMENT	VERSION
Node.js	≥ 18 LTS (tested: 20.19.2)
npm	≥ 9
Git	Any
Vercel account	For deployment

6.2 Local Development

```
# Install dependencies
npm install

# Start development server (port 3000, hot-reload)
npm start

# Production build (outputs to ./build/)
npm run build

# Run test suite
npm test
```

6.3 Production Build Output

```
build/
├─ index.html           (entry point – references hashed assets)
├─ asset-manifest.json
├─ static/
│   └─ css/
│       └─ main.[hash].css    (~9 kB gzip)
│   └─ js/
│       └─ main.[hash].js     (~249 kB gzip, includes React + jsPDF + html2canvas)
│       └─ [chunk].[hash].chunk.js (code-split chunks)
│       └─ *.LICENSE.txt
```

6.4 Vercel Deployment

```
# One-time: login to Vercel
npx vercel login

# Deploy to production
npx vercel --prod

# Deploy to preview URL
npx vercel
```

Vercel auto-detects Create React App; no `vercel.json` is required for basic deployment. However, `vercel.json` is required for iframe embedding (see Appendix B).

6.5 Alternative Deployment Targets

PLATFORM	NOTES
Netlify	Drop-in alternative to Vercel; same zero-config static site support
GitHub Pages	Free; requires <code>homepage</code> in <code>package.json</code> and <code>gh-pages</code> npm package
Docker + Nginx	For on-premise hosting; serve <code>build/</code> as static files
UWC Web Server	Contact ICT; provide the <code>build/</code> folder contents to serve as a static site

6.6 Environment Variables

No environment variables are required for the current version. If a backend is added (Phase 2), use:

```
REACT_APP_API_URL=https://api.example.com
```

CRA automatically injects `REACT_APP_*` variables at build time.

7. Security Architecture

7.1 Threat Surface Analysis

THREAT	LIKELIHOOD	IMPACT	MITIGATION
Malicious VPython code (XSS via iframe)	Low	Low	Sandboxed iframe; different origin; no DOM access to parent
CDN supply-chain attack	Very Low	High	Version-pinned URLs; SRI hashes (recommended)
localStorage injection	Very Low	None	No sensitive data in localStorage
Clickjacking via nested iframe	Low	Low	<code>frame-ancestors</code> CSP restricts who can embed
postMessage hijacking	Low	Low	Origin checking in trace message listener

7.2 iframe Sandbox

The `GLowCanvas` component embeds a cross-origin iframe pointing to `glowscript.org`. Because it is a **different origin**:

- The iframe **cannot access** the parent `window`, `document`, or `localStorage`.
- The parent **cannot access** the iframe's DOM (same-origin policy).
- Communication occurs only via `postMessage`, which is filtered by origin in the `useTrace` listener.

The iframe does not use the HTML `sandbox` attribute because GlowScript requires script execution. The cross-origin isolation itself provides equivalent isolation.

7.3 Content Security Policy

The recommended CSP for `vercel.json` (see Appendix B) allows:

```
default-src 'self'
script-src 'self' 'unsafe-inline' 'unsafe-eval' unpkg.com glowscript.org cdnjs.cloudflare.com
frame-src 'self' glowscript.org
style-src 'self' 'unsafe-inline' unpkg.com
img-src 'self' data: blob:
font-src 'self' data: unpkg.com
```

`'unsafe-inline'` and `'unsafe-eval'` are required by:

- Blockly (dynamic script evaluation)
- Monaco Editor (worker scripts)
- GlowScript's transpiler

Removing these would break the IDE. In a high-security environment, a `nonce`-based CSP could be implemented but would require server-side rendering.

7.4 Cross-Origin Resource Sharing (CORS)

Physics IDE makes no cross-origin API requests. All data is either:

- Local (`localStorage`)
- Loaded from CDN as script/style assets (CORS not applied to `<script src>`)
- Sent to the GlowScript iframe via `srcdoc` (no network request)

No CORS headers are needed on the Vercel deployment.

7.5 Subresource Integrity (SRI) — Recommended

To protect against CDN compromise, add `integrity` attributes to CDN `<script>` tags in `public/index.html`. SRI SHA-384 hashes can be computed at srihash.org.

```
<script
  src="https://unpkg.com/blockly@11/blockly_compressed.js"
  integrity="sha384- ..."
  crossorigin="anonymous"
></script>
```

The browser will refuse to execute the script if the content does not match the hash.

8. Performance Profile

8.1 Bundle Analysis

ASSET	SIZE (UNCOMPRESSED)	SIZE (GZIP)
Main JS bundle	~760 kB	~249 kB
CSS	~30 kB	~9 kB
CDN: Blockly v11	~3.5 MB	~1.1 MB
CDN: Monaco 0.45	~2.8 MB	~900 kB
CDN: GlowScript	Loaded lazily (on Run)	~400 kB

Total first load transfer: 2.7 MB (compressed). On a 10 Mbps connection this takes ~2 seconds. On campus eduroam (50–100 Mbps) this is effectively instant.

CDN libraries are heavily cached by the browser after the first visit. Subsequent loads transfer only the React bundle (~249 kB).

8.2 Runtime Performance

- **Block-to-code generation:** <5 ms for typical workspaces (synchronous, no async).
- **Auto-save:** Every 2 seconds; JSON.stringify of XML string (~100 ms worst case).
- **Trace updates:** Debounced at 50 ms; sparklines drawn via Canvas 2D API.
- **Simulation:** Runs in the GlowScript iframe; WebGL rendering performance depends on GPU. Typical frame rate: 60 fps on modern hardware, 20–30 fps on integrated GPUs.

8.3 Memory

- **localStorage quota:** Browsers allow 5–10 MB per origin. A typical block workspace XML is ~5–20 kB. No risk of quota exhaustion for educational use.
- **Trace history:** Rolling 60-point buffer per variable; gc'd automatically on `clearTrace`.

9. Sustainability & Maintenance

9.1 Dependency Longevity

DEPENDENCY	LIFECYCLE STATUS	RISK LEVEL	NOTES
React 18	Active	Low	Meta-maintained; React 19 is backward-compatible
Google Blockly v11	Active	Low	Google-maintained; used globally in education platforms
Monaco Editor 0.45	Active	Low	Microsoft-maintained (VS Code engine)
GlowScript 3.2	Community-maintained	Medium	Open-source; not commercially backed. Pinned at 3.2.
jsPDF 4.x	Active	Low	Widely used; open-source
html2canvas 1.4.x	Low activity	Medium	Last major release 2021; still functional
Create React App	Maintenance mode	Medium	No new features; migration to Vite recommended
Vercel hosting	Commercial, active	Low	Generous free tier; SLA for paid plans

9.2 CDN Pinning Strategy

All CDN URLs use explicit version numbers:

```
https://unpkg.com/blockly@11/blockly_compressed.js
https://unpkg.com/monaco-editor@0.45.0/min/vs/loader.js
```

This means:

- A CDN library update **never breaks the IDE** without a deliberate version bump.

- Security patches require manual version bumps (monitored via GitHub Dependabot or npm audit equivalent).

9.3 No-Backend Maintenance Model

The current architecture requires **zero server-side maintenance**:

- No patches for OS, web server, database, or runtime.
- No SSL certificate renewals (handled by Vercel).
- No capacity planning (Vercel auto-scales).
- **Total infrastructure cost:** \$0 (Vercel free tier covers educational traffic).

9.4 Code Maintainability

METRIC	ASSESSMENT
Source files	44 files across 8 logical layers
Largest file	<code>blocklyGenerator.js</code> (~1,600 LOC) — consider splitting by block category
CSS	Single <code>styles.css</code> (~800 LOC) — consider CSS Modules migration
Test coverage	No automated tests currently — add React Testing Library tests
Documentation	Inline JSDoc comments; this document

9.5 Recommended Migration Timeline

MILESTONE	ACTION	EFFORT
Q2 2026	Add SRI hashes to CDN scripts	2 hours
Q2 2026	Add <code>vercel.json</code> for iframe/CSP headers	1 hour
Q3 2026	Vite migration (retire CRA)	4–8 hours
Q4 2026	CSS Modules migration	1–2 days
Q1 2027	LTI 1.3 serverless endpoint (if grade integration needed)	3–5 days
Q1 2027	Per-user cloud save (Supabase)	1–2 weeks

10. UWC iKamva Sakai Integration

10.1 iKamva Overview

The **University of the Western Cape (UWC)** operates **iKamva** — its institutional Sakai LMS instance — at:

```
https://ikamva.uwc.ac.za/portal
```

iKamva provides the standard Sakai toolset: Resources, Assignments, Gradebook, Forums, Tests & Quizzes, Lessons, Web Content, and External Tools (LTI). The Faculty of Natural Sciences is the primary target faculty for Physics IDE integration.

10.2 Integration Option 1 — Web Content Tool (Recommended First Step)

Complexity: Low | **Development effort:** 2–4 hours | **Backend required:** No

The **Web Content** tool in Sakai embeds an external URL in an iframe within a course site's navigation. This is the simplest path to deploying Physics IDE within iKamva.

Configuration steps (for instructor / e-learning coordinator):

1. In the course site, go to **Site Info** → **Manage Tools** → **Web Content** → enable.
2. Set the URL to `https://physics-ide.vercel.app` (or your custom domain).
3. Set a display title (e.g., "Physics Simulator").
4. Save. The tool appears in the left navigation bar.

Required server-side configuration:

The Physics IDE Vercel deployment must return:

```
Content-Security-Policy: frame-ancestors https://ikamva.uwc.ac.za
X-Frame-Options: ALLOWALL
```

Without these headers, Sakai's browser security policy will block the iframe load. See Appendix B for the full `vercel.json`.

10.3 Integration Option 2 — External Tool (LTI 1.1)

Complexity: Medium | **Development effort:** 1–3 days | **Backend required:** Yes (minimal)

LTI 1.1 (Basic LTI) passes a signed POST request to the tool URL at launch time, containing:

- `user_id` — Sakai user identifier

- `roles` — "Instructor" or "Student"
- `context_id` — course site ID
- `oauth_signature` — HMAC-SHA1 request signature

A lightweight serverless function (e.g., Vercel Edge Function) can validate the signature and redirect to the Physics IDE SPA with the user context in URL parameters or a short-lived token.

This enables:

- Identifying which student is using the tool.
- Potentially routing students to different simulation templates based on course.
- Logging (if a persistent store is added).

10.4 Integration Option 3 — External Tool (LTI 1.3 Advantage)

Complexity: High | **Development effort:** 1–2 weeks | **Backend required:** Yes

LTI 1.3 uses OpenID Connect (OIDC) for the launch flow and replaces HMAC-SHA1 with RS256 JWTs. Additional LTI 1.3 services:

SERVICE	FUNCTION
Assignment & Grades Service (AGS)	Write back a score to the Sakai gradebook
Names & Roles Provisioning (NRPS)	Fetch the full roster of a course
Deep Linking	Instructor selects a specific template/configuration from within Sakai

Sakai support: LTI 1.3 Advantage is supported from Sakai 20 onwards. iKamva's version supports LTI 1.3 (confirmed by the Sakai wiki entry "Adding an LTI 1.3 Advantage Tool - Sakai v25").

LTI 1.3 Architecture

Student clicks tool in iKamva

| Sakai sends OIDC login initiation request

|— POST https://physics-ide.vercel.app/lti/login

| Physics IDE serverless function:

|— Validates state parameter

|— Returns redirect to Sakai's OIDC authorize endpoint

| Sakai sends id_token (JWT)

|— POST https://physics-ide.vercel.app/lti/launch

| Serverless function:

|— Verifies JWT signature (Sakai's public JWKS)

|— Extracts user_id, course_id, role, custom params

|— Generates a short-lived session token

|— Redirects to SPA: https://physics-ide.vercel.app/?token=...

| Physics IDE SPA reads token, initialises with user context

See Appendix C for the LTI 1.3 registration checklist.

10.5 Network & Firewall Requirements

The following domains must be reachable from student and staff browsers on campus and eduroam:

DOMAIN	PORT	PURPOSE
<code>physics-ide.vercel.app</code> (or custom domain)	443	Physics IDE application
<code>unpkg.com</code>	443	Blockly v11 + Monaco Editor CDN
<code>www.glowscript.org</code>	443	VPython 3.2 simulation runtime
<code>cdnjs.cloudflare.com</code>	443	Optional: additional library fallback

Action required: Submit a firewall whitelist request to UWC ICT for these domains. eduroam policies at UWC typically allow general HTTPS traffic, but `unpkg.com` and `glowsript.org` should be explicitly confirmed.

10.6 Bandwidth Impact

SCENARIO	DATA TRANSFERRED	FREQUENCY
First load (uncached)	~2.7 MB compressed	Once per browser
Subsequent loads (cached)	~249 kB (React bundle) + ~50 kB (app cache)	On each session
Running a simulation	~0 (GlowScript runs in iframe, no network after load)	Per run

For 30-student labs: **first load = $30 \times 2.7 \text{ MB} = 81 \text{ MB}$** in the first session; subsequent loads = ~8.7 MB per lab. Well within campus network capacity.

11. Data Privacy & POPIA

11.1 Current Data Handling

Physics IDE **collects and processes no personal data**. All data state is:

DATA	WHERE STORED	ACCESSIBLE TO SERVER?
Simulation workspace XML	<code>localStorage</code> in browser	No
Python code	<code>localStorage</code> in browser	No
Theme preference	<code>localStorage</code> in browser	No
Filename	<code>localStorage</code> in browser	No
Trace recordings	<code>localStorage</code> / in-memory	No

No user identifiers, IP addresses, or telemetry are captured or transmitted.

11.2 POPIA Compliance (Current Version)

South Africa's **Protection of Personal Information Act (POPIA)**, which came into full effect in July 2021, requires responsible parties to justify the processing of personal information. Since the current version processes **no personal information**, POPIA obligations are met by default.

11.3 POPIA Considerations for LTI Integration

If LTI integration is implemented and Physics IDE receives Sakai user identifiers:

OBLIGATION	ACTION
Lawful basis	LTI is processed under the UWC–student contract (educational purpose)
Data minimisation	Request only <code>user_id</code> and <code>role</code> — not email, name, or demographics
Purpose limitation	Use <code>user_id</code> only to identify saved workspaces
Retention	Delete user workspace data when a course ends or on request
Data processing agreement	Execute a DPA with any third-party storage provider (e.g., Supabase)

UWC's existing Sakai data processing agreements cover the LMS platform itself. Physics IDE would need to be registered as a processing activity in UWC's Information Officer records.

12. Accessibility

12.1 Current Status

AREA	WCAG 2.1 AA	STATUS	NOTES
Keyboard navigation	Required	Partial	Toolbar, modals keyboard-accessible; Blockly drag is mouse-only
Focus management	2.4.3	Partial	Focus not trapped in modals correctly
ARIA labels	1.3.1	Partial	Toolbar buttons labelled; trace table and debug panel need <code>aria-live</code>
Colour contrast	1.4.3	Pass (dark); TBD (light)	Run contrast check on light theme
Zoom/reflow	1.4.10	Partial	Layout breaks at <768px width
Screen reader	4.1.3	Minimal	3D canvas not screen-reader accessible (inherent WebGL limitation)

12.2 Sakai Accessibility Requirements

Sakai's own accessibility commitment (WCAG 2.1 AA) requires that embedded tools meet the same standard. Physics IDE should be audited with:

- [axe DevTools](#) (browser extension)
- [NVDA](#) or VoiceOver screen reader testing

- Keyboard-only navigation test

12.3 WebGL Accessibility Limitation

The VPython simulation viewport renders entirely in WebGL via an iframe. WebGL content is **not accessible to screen readers** by design — it is a visual presentation medium. Mitigation:

- Provide text descriptions of simulation outcomes (the `label()` and `scene.caption` VPython API already prints text output).
- Provide alternative data formats (trace table CSV export provides the numerical results).
- Document this limitation in the tool's accessibility statement.

13. Recommended Roadmap

Phase 0 — Immediate (Before Semester Integration)

TASK	OWNER	EFFORT
Create <code>vercel.json</code> with iframe headers	Developer	1 hour
Deploy to Vercel; set custom domain	Developer	2 hours
Whitelist CDN domains with UWC ICT	ICT / Developer	1 day
Test embedding in iKamva Web Content tool	e-Learning Coordinator	Half day
Confirm with Faculty of Natural Sciences on course integration	Academic	As needed

Phase 1 — Before Formal Classroom Use

TASK	OWNER	EFFORT
WCAG contrast audit (light theme)	Developer	2 hours
Add <code>aria-live</code> regions to trace table & debug panel	Developer	4 hours
Add focus trap to modals (VariableDialog, HelpPage)	Developer	2 hours
SRI hashes on CDN scripts	Developer	2 hours
Educator onboarding documentation	Author	1 day

Phase 2 — Enhancement (Semester 2 or 2027)

TASK	OWNER	EFFORT
Vite migration (retire CRA)	Developer	4–8 hours
CSS Modules (per-component styles)	Developer	1–2 days
LTI 1.3 serverless launch handler	Developer	3–5 days
Per-user workspace persistence (Supabase)	Developer	1 week
Self-hosted CDN assets (offline support)	Developer	1–2 days
Mobile-responsive layout	Developer	2–3 days

Phase 3 — Long-Term

TASK	BENEFIT
i18n support (Afrikaans, isiXhosa)	Accessibility for multilingual student body
Grade passback (AGS)	Formal assessment integration with iKamva gradebook
Block library expansion	More VPython objects (compound shapes, lights, materials)
Collaborative editing	Real-time multi-student simulation editing

14. Appendix A — Constants Reference

```
// src/constants/index.js
STORAGE_KEY      = "physide_workspace"    // localStorage key for auto-save
THEME_STORAGE_KEY = "physide_theme"        // localStorage key for theme
DEFAULT_PYTHON_CODE = "GlowScript 3.2 VPython\n# Start writing...\n"
ZOOM_MIN          = 15                    // % - minimum zoom slider value
ZOOM_MAX          = 100                   // % - maximum zoom slider value
ZOOM_DEFAULT      = 50                    // % - initial zoom
SPLIT_MIN         = 15                    // % - minimum editor panel width
SPLIT_MAX         = 85                    // % - maximum editor panel width
SPLIT_DEFAULT     = 50                    // % - initial split position
TRACE_HISTORY_SIZE = 60                   // data points retained per variable
TRACE_DEBOUNCE_MS = 50                    // ms - trace update debounce interval
HIGHLIGHT_DURATION_MS = 250               // ms - block highlight duration
AUTOSAVE_INTERVAL_MS = 2000               // ms - auto-save interval
GLOWSCRIPT_HOST_ID = "glowscript-host"    // iframe element ID
```

15. Appendix B — vercel.json Template

Create this file in the project root before deploying:

```
{
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        {
          "key": "Content-Security-Policy",
          "value": "default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval' htt
        },
        {
          "key": "X-Frame-Options",
          "value": "ALLOWALL"
        },
        {
          "key": "X-Content-Type-Options",
          "value": "nosniff"
        },
        {
          "key": "Referrer-Policy",
          "value": "strict-origin-when-cross-origin"
        },
        {
          "key": "Permissions-Policy",
          "value": "camera=(), microphone=(), geolocation=()"
        }
      ]
    }
  ]
}
```

Security note: `frame-ancestors https://ikamva.uwc.ac.za` restricts embedding to iKamva only. To additionally allow testing from localhost, add `https://localhost:3000` :

```
frame-ancestors https://ikamva.uwc.ac.za https://localhost:3000
```

16. Appendix C — LTI 1.3 Registration Checklist

When registering Physics IDE as an LTI 1.3 tool in iKamva:

Information to provide to Sakai admin (iKamva):

FIELD	VALUE
Tool Name	Physics IDE
Description	3D physics simulation environment for VPython/GlowScript
Launch URL	<code>https://physics-ide.vercel.app/lti/launch</code>
Login Initiation URL	<code>https://physics-ide.vercel.app/lti/login</code>
Redirect URI(s)	<code>https://physics-ide.vercel.app/lti/launch</code>
JWKS URL (public key)	<code>https://physics-ide.vercel.app/.well-known/jwks.json</code>
Deep Linking URL	<code>https://physics-ide.vercel.app/lti/deeplink</code> (optional)
Privacy level	<code>Name only</code> (or <code>Anonymous</code> if no user identification is needed)
LTI Advantage Services	AGS: optional; NRPS: optional

Information received from Sakai admin:

FIELD	DESCRIPTION
Platform Issuer	e.g., <code>https://ikamva.uwc.ac.za</code>
Client ID	Assigned by Sakai on tool registration
Platform OIDC Auth URL	Sakai's OIDC authorize endpoint
Platform JWKS URL	Sakai's public key endpoint (to verify id_token)
Deployment ID	Assigned per deployment (course site)

Environment variables for the serverless function:

```
LTI_ISS=https://ikamva.uwc.ac.za
LTI_CLIENT_ID=<from_sakai>
LTI_PLATFORM_JWKS_URL=https://ikamva.uwc.ac.za/.well-known/jwks.json
LTI_PLATFORM_OIDC_URL=https://ikamva.uwc.ac.za/imsblis/lti13/oidc
LTI_PRIVATE_KEY=<RS256 private key in PEM format>
```

